

SDN-Based Access Control System

Project Documentation & Screenshot Analysis

Samyuktha V | PES1UG24AM312 | UE24CS252B — Computer Networks | PES University

1. Project Overview

This project implements an SDN (Software Defined Networking) based Access Control System using Mininet and the Ryu OpenFlow 1.3 controller. The system allows only authorized (whitelisted) hosts to communicate, while all unauthorized hosts are automatically blocked by controller-defined flow rules.

Component	Details
Hosts	h1 (10.0.0.1), h2 (10.0.0.2), h3 (10.0.0.3)
Switch	s1 — Open vSwitch (OpenFlow 1.3)
Controller	Ryu Controller (access_control.py)
Whitelist (ALLOWED)	h1 <-> h2 only
Blocked	h3 — all traffic dropped

2. Screenshot 1 — Mininet Topology & Ping Test Results

What is this?

This is Terminal 2 — the Mininet terminal. It shows the virtual network being created and two ping tests being run: one allowed (h1 to h2) and one blocked (h1 to h3).

```
seed@seedvm2004: ~/sdn_proj
seed@seedvm2004: ~$ sudo mn --controller=remote,ip=127.0.0.1,port=6633 --topo=single,3 --sw
tch ovsk,protocols=OpenFlow13
[sudo] password for seed:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=9.33 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=2.78 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=8.62 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.781/6.910/9.328/2.934 ms
mininet> h1 ping -c3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data:

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2032ms

mininet>
```

Screenshot 1: Mininet terminal showing network creation and ping test results

Line-by-Line Explanation:

➤ **sudo mn --controller=remote ... --topo=single,3**

This command starts Mininet with a remote Ryu controller, creates 3 hosts (h1, h2, h3) and 1 switch (s1) using OpenFlow 1.3.

➤ **Adding hosts: h1 h2 h3 / Adding switches: s1**

Mininet creates 3 virtual computers and 1 virtual switch in memory.

➤ **Adding links: (h1,s1) (h2,s1) (h3,s1)**

Each host is connected to the switch via a virtual Ethernet cable (veth link).

➤ **Starting controller c0**

The switch connects to the Ryu controller running on localhost port 6633.

➤ **h1 ping -c3 h2 → 0% packet loss**

h1 sends 3 pings to h2. Since h1 and h2 are whitelisted, the controller ALLOWS all packets. Result: 3/3 received, 0% loss. SUCCESS!

➤ **h1 ping -c3 h3 → 100% packet loss**

h1 tries to ping h3. Since h3 is NOT whitelisted, the controller DROPS all packets. Result: 0/3 received, 100% loss. BLOCKED!

Test	Command	Result	Status
Authorized ping	h1 ping -c3 h2	0% packet loss	✓ ALLOWED

Switch s1 has successfully connected to the Ryu controller over localhost. They are now communicating!

► hello ev — EventOFPHello

The switch sends a Hello message to start the OpenFlow handshake. This is normal protocol behavior.

► move onto config mode → move onto main mode

Controller goes through OpenFlow setup: first installs default rules (config), then handles live traffic (main).

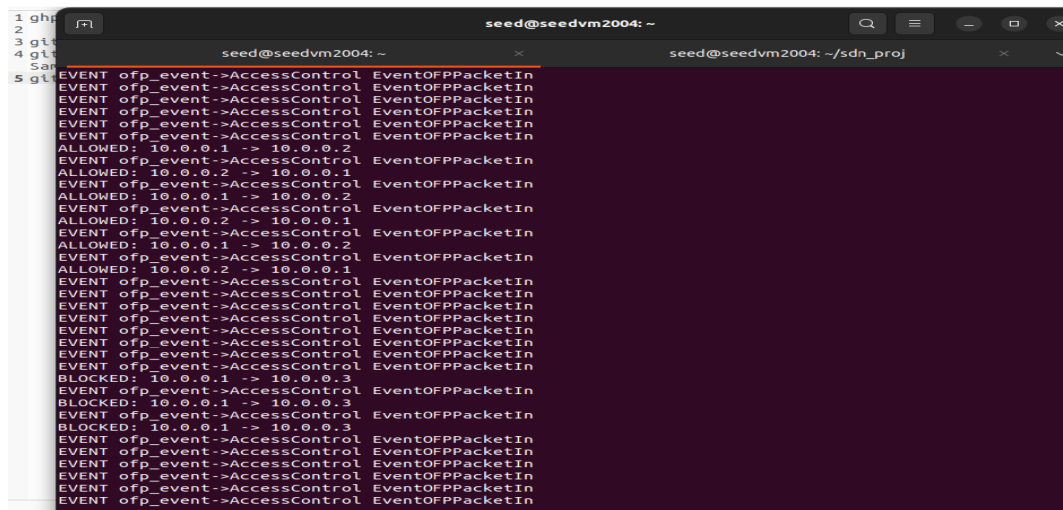
► EVENT ofp_event->AccessControl EventOFPPacketIn (repeated)

Each time a packet arrives with no matching flow rule, it gets sent to our controller for a decision.

4. Screenshot 3 — Ryu Controller ALLOWED & BLOCKED Logs

What is this?

This is Terminal 1 again (Ryu controller) but now showing real-time logs during the ping tests. This is the most important screenshot — it proves the access control policy is working exactly as intended.



```
seed@seedvm2004: ~  
seed@seedvm2004: ~/sdn_proj  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
ALLOWED: 10.0.0.1 -> 10.0.0.2 EventOFPPacketIn  
ALLOWED: 10.0.0.2 -> 10.0.0.1 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
ALLOWED: 10.0.0.1 -> 10.0.0.2 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
ALLOWED: 10.0.0.2 -> 10.0.0.1 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
ALLOWED: 10.0.0.1 -> 10.0.0.2 EventOFPPacketIn  
ALLOWED: 10.0.0.2 -> 10.0.0.1 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
BLOCKED: 10.0.0.1 -> 10.0.0.3 EventOFPPacketIn  
BLOCKED: 10.0.0.1 -> 10.0.0.3 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
BLOCKED: 10.0.0.1 -> 10.0.0.3 EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn  
EVENT ofp_event->AccessControl EventOFPPacketIn
```

Screenshot 3: Ryu controller logs showing ALLOWED and BLOCKED decisions in real time

Line-by-Line Explanation:

► ALLOWED: 10.0.0.1 -> 10.0.0.2

h1 is sending a packet to h2. The controller checks the whitelist, finds this pair, and ALLOWS it. The packet is forwarded to h2. This appears multiple times — once per ping packet.

► ALLOWED: 10.0.0.2 -> 10.0.0.1

This is the return traffic — h2 replies back to h1. Both directions are whitelisted so the reply is also allowed. This confirms two-way communication works.

► **BLOCKED: 10.0.0.1 -> 10.0.0.3**

h1 tries to reach h3. The controller checks the whitelist, doesn't find this pair, and DROPS the packet. h3 never receives anything. Appears 3 times — once per ping attempt.

► **Why no BLOCKED: 10.0.0.3 -> 10.0.0.1?**

Since h3 never receives any packets (all dropped), h3 never sends a reply. So there is no reverse traffic to block — the blocking is complete and clean.

► **EVENT ofp_event->AccessControl EventOFPPacketIn (continued)**

After the blocked pings, the controller continues listening for more packets. It stays running and ready for any future traffic.

Access Control Policy Summary:

Source	Destination	Controller Decision	Reason
h1 (10.0.0.1)	h2 (10.0.0.2)	ALLOWED ✓	In whitelist
h2 (10.0.0.2)	h1 (10.0.0.1)	ALLOWED ✓	In whitelist
h1 (10.0.0.1)	h3 (10.0.0.3)	BLOCKED ✗	Not in whitelist
h3 (10.0.0.3)	h1 (10.0.0.1)	BLOCKED ✗	Not in whitelist